

Notes

Joaquin Arias

March 7, 2024

1 Comprehensive Overview of Functions and Parameters

04/03/2024

1.1 Commulative Reward

The reward determines how good the action from the state s to reach the next state. This is the crucial component of RL that determines the learning of the RL agent. On the other hand, the return is the cumulative sum of weighted rewards from the current state to the goal state.

$$R(\tau) = r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} + \dots$$

$$R(\tau) = \sum_{k=0}^{\infty} \gamma^k r_{t+k+1}$$

Where τ is the trajectory (sequence of states and actions) and the discount rate is $0 \leq \gamma \leq 1$ (usually γ is 0.95 or 0.99). Large $\gamma \Rightarrow$ less discount $\Rightarrow$ agent cares more about long-term rewards. Smaller $\gamma \Rightarrow$ more discount $\Rightarrow$ agent cares more about short-term rewards [2].

1.2 Learning Details

Proposed Model Sketch:

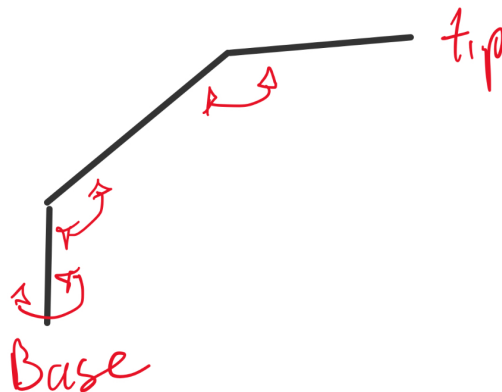


Figure 1: Model Sketch.

Action Space:

- Rotation of the panning shoulder.
- Rotation of the shoulder lifting joint.
- Rotation of the forearm flexing joint.

An action (a, b) represents the torques applied at the joints.

Observation Space:

- Angle of the joints on the arm (3).
- Angular velocities of these joints (3).
- Coordinates of the tip of the arm (3).
- Coordinates of the object to be pushed (3).
- Coordinates of the goal (3).

This gives us a 'ndarray' observation space of shape '(15,)'

Rewards:

- reward_near: measure of how far the tip is from the object.

$$reward_near = -\|P_{tip} - P_{object}\|$$

- reward_dist: measure of how far the object is from the goal.

$$reward_dist = -\|P_{object} - P_{goal}\|$$

- reward_speed: measure of how fast the reaches the goal.

$$reward_speed = \frac{reward_dist_{t-1} - reward_dist_t}{\Delta t}$$

- reward_control: negative reward if the robot takes too large actions.

$$reward_control = -\|a\|^2 = -\sum_{i=1}^n a_i^2$$

Where a_i is the i -th component of the action vector and n is the dimensionality of the action space.

Training:

In QD methods, solutions $x_i \in \chi$ encode parameters for policies π_{x_i} for an agent that interacts in a sequential environment at discrete time-steps t . Each time-step:

$$agent \xrightarrow[\text{and chooses action } a_t]{\text{observes } s_t} s_{t+1}, r_t$$

The policy determines which action is taken in each time-step $\mathbb{P}[a_t|s_t]$. To measure how good this policy is, we define an objective function $J(x)$ which outputs the expected cumulative reward of the agent over the whole time frame (T time-steps):

$$J(x) = \sum_{j=0}^T \gamma^j r(s_j, a_j)$$

To improve the optimization process, as solutions are evaluated, transition tuples $(s_t, a_t, s_{t+1}, a_{t+1})$ are stored in a replay buffer $\mathcal{B}$. With these, we train a critic network that approximates:

$$Q(s_t, a_t) = \mathbb{E} \left[\sum_{j=t}^T \gamma^j r(s_j, a_j) \right],$$

which is the action-value function that captures the value of taking an action a_t from the state s_t . These critic estimates can be used to form a policy gradient estimate:

$$\nabla_{x_i} J_{x_i} = \mathbb{E} \left[\nabla_{x_i} \pi_{x_i}(s) \nabla_a Q_\theta(s, a) |_{a=\pi_{x_i}(s)} \right],$$

where Q_θ is the critic network [3]. Here we can see that the policy gradient provides the direction in which a solution x_i should be updated in order to maximize the reward of the agent. In each iteration, some solutions are mutated via gradient-based updates and, to maintain exploration, others are mutated via genetic mutations [1].

1.3 Why Quality-Diversity:

To better understand the advantages of QD algorithms over traditional methods, consider the task of training a robot to grasp objects of various shapes and sizes while avoiding collisions with obstacles. The task involves multiple conflicting objectives, such as maximizing the success rate

of grasping objects, minimizing the time taken to complete a task, and avoiding collisions. Also the environment is highly dynamic, with objects being added or removed over time. PPO focuses on optimizing a single policy to maximize the cumulative reward. However, it can get stuck at the local optima, without really exploring other potentially better solutions. With MOME-PGX instead, we can explore a wide range of grasping strategies and object interactions, discovering diverse solutions. We can also optimize multiple objectives.

References

- [1] Hannah Janmohamed, Thomas Pierrot, and Antoine Cully. Improving the data efficiency of multi-objective quality-diversity through gradient assistance and crowding exploration. In *Proceedings of the Genetic and Evolutionary Computation Conference*, GECCO '23. ACM, July 2023.
- [2] T. Simonini. Deep rl course. 2018.
- [3] Cheng Xi Tsou. Introduction to deterministic policy gradient (dpg), Aug 2021.